

Lesson 6: Theming Your App

One place for all your styles

Look at any good app. The buttons, the headings, the colors -- they all match. That does not happen by styling each widget by hand. It happens because the app has a theme: one place where all the shared styles live.

In Flutter, the theme lives on your `MaterialApp`. You set it once, and every screen below it can use it.

Setting a theme

The `MaterialApp` has a `theme` property. You give it a `ThemeData` object, which holds your colors, fonts, and shapes:

```
MaterialApp(  
  title: 'My App',  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
    fontFamily: 'Roboto',  
  ),  
  home: const HomeScreen(),  
);
```

With just these two lines, every `AppBar` in your app turns blue, and all text uses Roboto. You did not touch a single screen to make that happen.

ThemeData: the style box

`ThemeData` is like a box full of named styles. The most useful ones are:

- `primarySwatch` -- the main color of your app. Buttons, app bars, and highlights use it.
- `fontFamily` -- the default font for all text.
- `scaffoldBackgroundColor` -- the background color of every `Scaffold`.
- `brightness` -- `Brightness.light` or `Brightness.dark`. This tells Flutter whether your theme is a light one or a dark one, so it can pick readable default text colors.

```
theme: ThemeData(  
  primarySwatch: Colors.green,  
  scaffoldBackgroundColor: Colors.grey[100],  
  brightness: Brightness.light,  
),
```

Change one value here, and the whole app updates. That is the power of a theme: one change, everywhere.

Reading the theme with Theme.of(context)

Widgets often need to know the current theme -- for example, to paint something in the primary color. They read it with Theme.of(context):

```
Container(  
  color: Theme.of(context).primaryColor,  
  child: Text(  
    'Hello',  
    style: Theme.of(context).textTheme.headlineMedium,  
  ),  
)
```

Theme.of(context) walks up the widget tree, finds the nearest Theme (which MaterialApp puts there for you), and returns its ThemeData. This is the same context idea from Lesson 5: widgets find shared things above them through the context.

Why do this instead of writing Colors.blue directly? Because if you later change your theme to green, every widget that used Theme.of(context) updates on its own. Widgets with hard-coded colors stay blue -- and now your app looks broken.

Light mode and dark mode

Many apps let the user switch between a light theme and a dark theme. Flutter makes this easy: MaterialApp has a second property called darkTheme, plus a themeMode that picks which one to show.

```
MaterialApp(  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
    brightness: Brightness.light,  
  ),  
  darkTheme: ThemeData(  
    primarySwatch: Colors.blue,  
    brightness: Brightness.dark,  
  ),  
  themeMode: ThemeMode.system, // follows the phone's setting  
  home: const HomeScreen(),  
);
```

ThemeMode has three choices:

- ThemeMode.light -- always use the light theme.
- ThemeMode.dark -- always use the dark theme.
- ThemeMode.system -- follow what the user picked in their phone settings.

With `ThemeMode.system`, your app switches between light and dark on its own. You wrote two themes; Flutter handles the rest.

Key terms

- `Theme` -- the shared set of styles (colors, fonts, shapes) for your whole app.
- `ThemeData` -- the object that holds all the style values for a theme.
- `primarySwatch` -- the main color of your app, used by buttons and app bars.
- `Theme.of(context)` -- how a widget reads the current theme from above it in the tree.
- `darkTheme` -- a second theme used when the app is in dark mode.
- `ThemeMode` -- picks which theme to show: light, dark, or follow the phone.